

Name: Nibretu Mengaw

Id: CTC-406-26

Python Backend Development with Flask

Backend Development

Backend development is the invisible layer where data is processed, security is checked, and business logic is executed. While the frontend handles what the user sees, the backend focuses on how things work, managing databases, and making sure the right information gets sent to the right place and other business logics

Flask

Flask is a Python framework used to build the backend servers. What makes it unique is it doesn't force a rigid structure or heavy tools we don't need. It gives us a clean, simple starting point to handle web requests, allowing us to pull in extra tools (like databases) only when our project actually requires them.

Api

Api is like a messenger it takes a request and take back the responses as well. When we look at a Client-server communication the client (like a browser or mobile app) starts the conversation by asking for something, and the server processes that request and talks back. An API acts as the bridge or the translator between them.

HTTP Methods

HTTP methods are the verbs we use to tell the backend exactly what action we want to perform:

- ▶ **GET**: Used to safely fetch or read data from the server without changing anything.
- ▶ **POST**: Used to send new data to the server, like creating a new user account.
- ▶ **PUT**: Used to update or replace existing data that is already on the server.
- ▶ **DELETE**: Used to permanently remove specific data from the server.

Request and Response

The flow always start with a request from the client, which packs together a URL, an HTTP method and sometimes with data like form data (email, password for login). The server receives this package(request) and process it and sent back the Response and the response contains the status code like(200 for success and 404 for not found and other status code....) and the actual data the client asked for.

JSON (JavaScript Object Notation)

APIs use JSON. It formats the information in to simple text using key-value pairs that are lightweight to send over the internet. I think the reason of API is using JSON is also like almost every programming language can read it easily and it allows the backend talks to the frontend smoothly.

Routing in Flask

In the video one of the things I understand is the flask routing logic. It is about how the flask identify the specific web URL to a specific python function to perform what exactly the user wants. Flask uses decorators to identify which method should be executed when a specific API call happens it uses like this

```
@app.route("/home")
def home():
    return "This is the home page"
```

and when the user visits host/home it execute the home function and return the response.

What I Learned In General

The videos make it clear how all these separate things connected and form a working web application. And It seems like a simple python function turning in to a live URL endpoint that can receive data and process it or return a clean JSON response.

It showed me that building a backend is not about writing a complex code but it is about managing a clean flow of a requests and response.

- ***I learn not only by watching the video but also I was building by following the tutorial and I commit the project in the same folder as this pdf file inside week2/flask***

how to use the folder:

- cd to flask folder(cd flask)
- create virtual environment(python -m venv venv)
- activate it (e.g source venv/bin/activate for linux)
- install the packages like flask, flask_sqlalchemy (pip install flask,)
- and open the Python shell(inside flask folder run 'python') it will open python shell.
- import the db from app(from app import db)
- create db (db.create_all())
- then quit the shell and run the app.py file then visit the endpoint where the app runs and try to add tasks.